Object-oriented Programming

Week 9 | Lecture 1&2

default arguments in c++

- A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn’t provide a value for the argument with a default value.

Rule 1

- Default arguments must be the rightmost (trailing) arguments in a function's parameter list. default arguments are assigned from right to left
- Once we provide a default value for a parameter, all subsequent parameters must also have default values. For example,

- // Invalid
- void add(int a, int b = 3, int c, int d);

- // Invalid
- void add(int a, int b = 3, int c, int d = 4);

- // Valid
- void add(int a, int c, int b = 3, int d = 4);

Rule 2

- If we are defining the default arguments in the function definition instead of the function prototype, then the function must be defined before the function call.

```
// Invalid code

int main() {
    // function call
    display();
}

void display(char c = '*', int count = 5) {
    // code
}
```

Operator Overloading

- C++ allows you to specify more than one definition for an operator in the same scope, which is called operator overloading.
- You can redefine or overload most of the built-in operators available in C++
- It is a type of polymorphism in which an operator is overloaded to give user defined meaning to it.

Operator Overloading

- Defining a new behavior for common operators of a language
- C++ enables you to overload most operators to be sensitive to the context in which they're used
- Using operator overloading makes a program clearer than accomplishing the same operations with function calls

When operators are overloaded as member functions, they must be non-static

- When operators are overloaded as **member functions**, they must be **non-static** because they operate on **instances** of a class

Static Functions Do Not Have this

•A static function does not have access to this, meaning it cannot directly access instance variables.

•Since operator overloading usually involves modifying or accessing object attributes, a static function would not work properly.

Operator Overloading

- An operator is overloaded by writing a non-static member function definition or global function definition

- To use an operator on class objects (as operands), that operator **“must” be overloaded**

Operator Overloading

- Operator overloading cannot change the arity of an operator
- Operator overloading works when *at least* one argument of that operator is an object
- We cannot create new operators using operator overloading

LIST OF OPERATORS THAT CAN'T BE OVERLOADED

- Almost any operator can be overloaded in C++. However there are few operator which can not be overloaded.
- scope operator (::)
- sizeof
- member selector –(.)
- member pointer selector – (.*)
- ternary operator – (?:)

BUILT IN OVERLOADS

- Most operators are already overloaded for fundamental types.
- Example:
- 1) In the case of the expression:
- a / b the operand type determines the machine code created by the compiler for the division operator. If both operands are integral types, an integral division is performed;
- in all other cases floating-point division occurs. Thus, different actions are performed depending on the operand types involved.
- 2) <<, which is used both as the stream insertion operator and as the bitwise left-shift operator.

Works fine

```
#include <iostream>
using namespace std;
int main() {
        int a=5;
        int b=3;
        int z=a+b;
        cout << z;
     return 0;}
```

Output?

```
#include <iostream>
using namespace std;
 class Complex {
   private:
      int real;
      int image;
   public:
Complex(){
        real = 0;
        image = 0; }
Complex(int r, int i){
        real = r;
        image = i;}
```

Output?

- void displayComplex() {

```
 cout << "real: "<< real << " Imaginary:" <<image <<endl;     }};
int main() {
        Complex c1(2,1);
        Complex c2(3,1);
        Complex c3;
        c3=c1+c2;
    return 0;}
```

Output

- [Error] no match for 'operator+' (operand types are 'Complex' and 'Complex')

criteria/rules to define the operator function:

- In case of a non-static function, the binary operator should have only one argument and unary should not have an argument.

Syntax of Operator Overloading

- Return type operator op(argument list);

- return_type class_name : : operator op(argument_list)
- {
- // function body
- }

Example of Operator Overloading

```
class Complex {
   private:
      int real;
      int image;
   public:
Complex(){
        real = 0;
        image = 0; }
Complex(int r, int i){
        real = r;
        image = i;}
```

Example of Operator Overloading

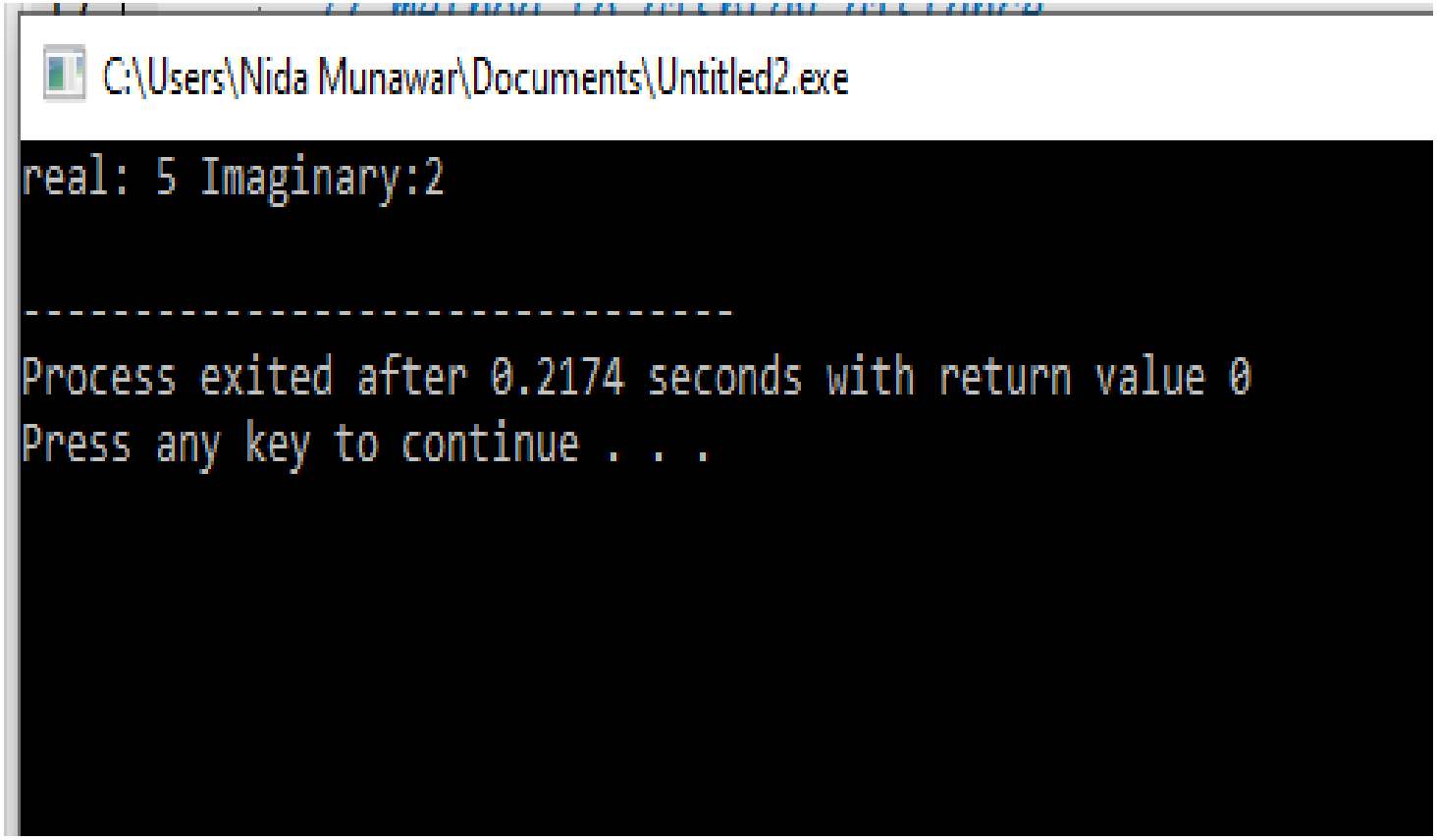
```
void displayComplex() {
 cout << "real: "<< real << " Imaginary:" <<image
<<endl;     }
// overloaded  (+) operator
      Complex operator+ (Complex c) {
          Complex temp;
        temp.real=real+c.real;
        temp.image=image+c.image;
        return temp;
      }};
```

Example of Operator Overloading

```
int main() {
        Complex c1(2,1);
        Complex c2(3,1);
        Complex c3;
        c3 = c1.operator+ ( c2 );
         //c3=c1+c2;
        c3.displayComplex();
   return 0;}
```

- c3=c1+c2; // statement 1
- It is same like
- c3=c1.add(c2); just for understanding
- In statement 1, Left object c1 will invoke operator+() function and right object c2 is passing as an argument.
- Another way of calling binary operator overloading function is to call like a normal member function as follows,

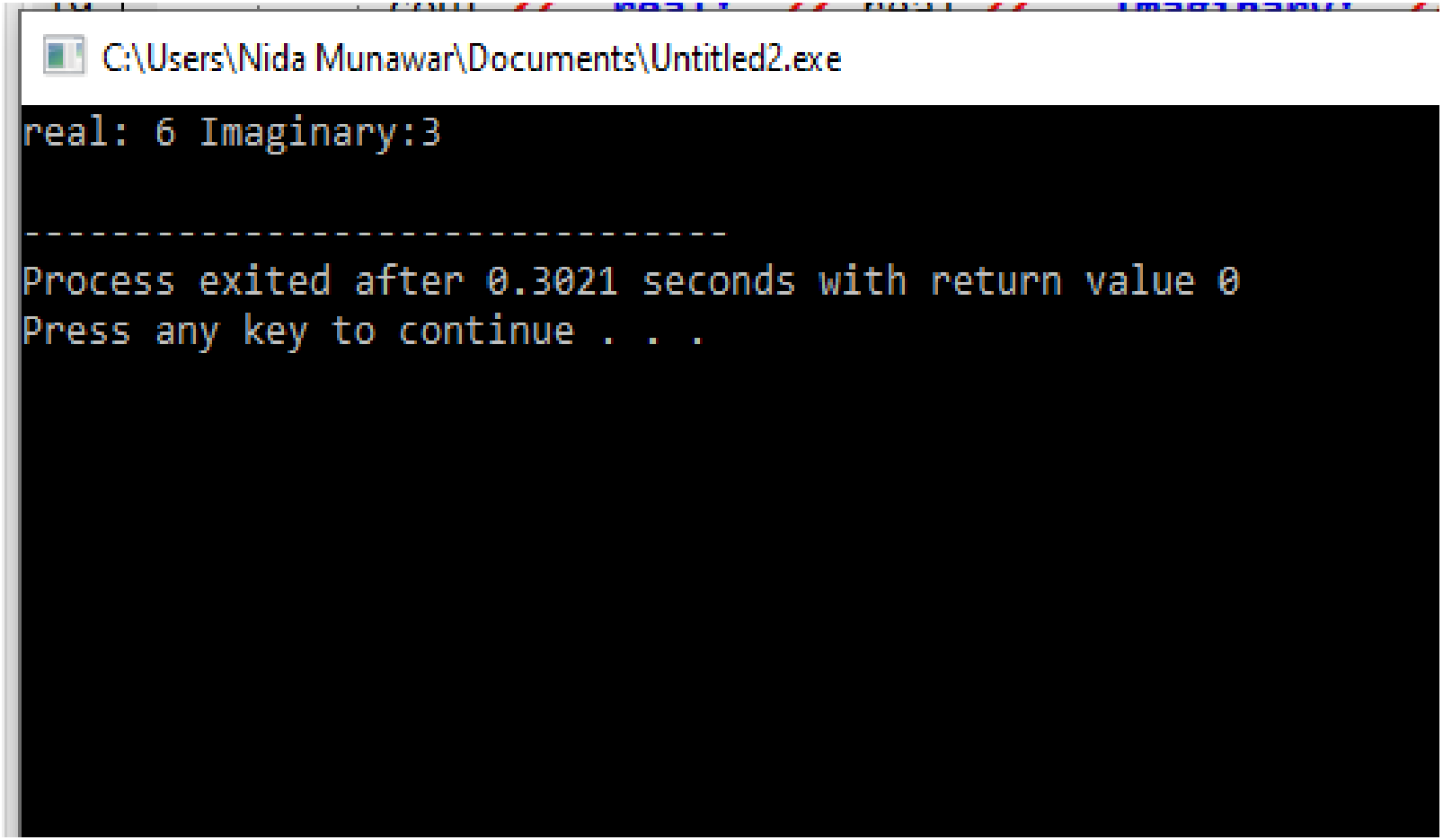
c3 = c1.operator+ (c2);


C:\Users\Nida Munawar\Documents\Untitled2.exe
real: 5 Imaginary:2

Process exited after 0.2174 seconds with return value 0
Press any key to continue . . .

```
int main() {
        Complex c1(2,1);
        Complex c2(3,1);
        Complex c3(1,1);
        Complex c4;
        c4=c1+c2+c3;
        c4.displayComplex();
      return 0;}
```

- c4=c1+c2+c3;
- Firstly addition of c1+c2 happen then the result of c1+c2 added to c3 and stored in c4 here add function called two times


C:\Users\Nida Munawar\Documents\Untitled2.exe
real: 6 Imaginary:3

Process exited after 0.3021 seconds with return value 0
Press any key to continue . . .

```
#include <iostream>
using namespace std;
class Binary {
   int i;
public:
   Binary(int val = 0) : i(val) {}

   // Binary + Operator Overload (returns new object)
   Binary operator+(const Binary& obj) {
      Binary temp;  // Create a new object (temp)
      temp.i = this->i + obj.i;
      return temp;  // Return new object (not reference)
   }

   void Display() const {
      cout << "i=" << i << endl;
   }
};
```

```
int main() {
    Binary obj1(5), obj2(3), obj3;

    obj3 = obj1 + obj2;  // obj3 = obj1.operator+(obj2);
    obj3.Display();  // Output: i=8

    Binary obj4;
    obj4 = obj1 + obj2 + obj3;  // Chaining works: obj4 = (obj1 + obj2) + obj3;
//obj4 = obj1.operator+(obj2).operator+(obj3);

    obj4.Display();  // Output: i=16

    return 0;
}
```

Step-by-Step Execution of obj1 + obj2 + obj3

1. First, obj1 + obj2
   - temp1.i = obj1.i + obj2.i = 5 + 3 = 8
   - Returns a new object (temp1 with i=8).
2. Then, temp1 + obj3
   - temp2.i = temp1.i + obj3.i = 8 + 8 = 16
   - Returns another new object (temp2 with i=16).
3. Assign result to obj4
   - obj4 = temp2
   - obj4.i = 16

Step-by-Step Breakdown:

1.obj1 + obj2 → Calls obj1.operator+(obj2), returns a new object (say temp1).

2.temp1 + obj3 → Calls temp1.operator+(obj3), returns another new object (say temp2).

3.obj4 = temp2 → Assigns temp2 to obj4.

Prefix Increment (++i)

- Increments first, then returns the updated value.

```
#include <iostream>
int main() {
    int i = 5;
    int x = ++i;

    std::cout << "i = " << i << std::endl;
    std::cout << "x = " << x << std::endl;}
```

Postfix Increment (i++)

- Returns the original value first, then increments.

```
#include <iostream>
int main() {
    int i = 5;
    int x = i++;

    std::cout << "i = " << i << std::endl;
    std::cout << "x = " << x << std::endl;
}
```

Difference between post and prefix

- int y=0;
- y=x++;
- cout <<x;
- cout<<y;

- X=1
- Y=0
- In postfix firstly substitute the value then increment happens

For Prefix ++ operator

```
void operator ++ ( )
{
        ++x;
        ++y;
}
```

(Works the same way for prefix decrement operator)

For Prefix ++ operator

```
class Prefix{
      int i;
   public:
      Prefix(): i(0) {  }
      void operator ++()
        { ++i; }
      void Display()
        { cout << "i=" << i << endl; }};
int main(){
   Prefix obj;
   obj.Display();
   ++obj;
   //you can also write  obj.operator ++();
   obj.Display();
         return 0;}
```

- Prefix (++obj) works by first incrementing and then returning the object.
- Returning *this allows chaining (++(++obj);).

```
class Prefix {
    int i;
public:
    Prefix() : i(0) {}

    // Prefix increment overloading
    Prefix& operator++() {  // Returning reference
        ++i;
        return *this;  // Return the updated object
    }

    void Display() {
        cout << "i=" << i << endl;
    }
};
```

```
int main() {
    Prefix obj;
    obj.Display();  // i = 0

    ++obj;  // Calls operator++()
    obj.Display();  // i = 1

    ++(++obj);  // Works because we return a reference
// obj.operator++().operator++();
    obj.Display();  // i = 3

    return 0;
}
```

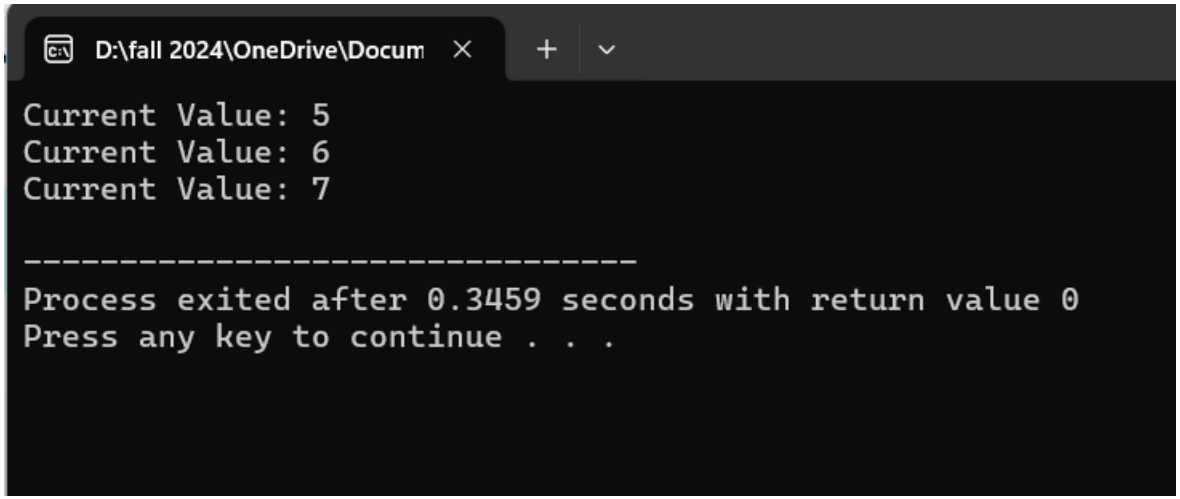
execution order

This means that the execution order follows left-to-right evaluation:
1.obj.operator++() → modifies obj (increases value by 1)
2.obj.operator++() (on the modified obj) → modifies obj again
(increases value by 1 more)

Output?

```
#include <iostream>
class Number {
    int value;
public:
    Number(int v) : value(v) {}
    Number(){ }
    Number operator++() {
        Number z;
         z= ++value;
        return z;  }
    void display() const {
std::cout << "Current Value: " << value << std::endl;
    }};
```

```
int main() {
    Number obj(5);
    obj.display();
    Number o;
    o=++(++obj);  //obj.operator++().operator++();
    obj.display();
    o .display();//effects on z object
    return 0;
}
```


D:\fall 2024\OneDrive\Docum
Current Value: 5
Current Value: 6
Current Value: 7

Process exited after 0.3459 seconds with return value 0
Press any key to continue . . .

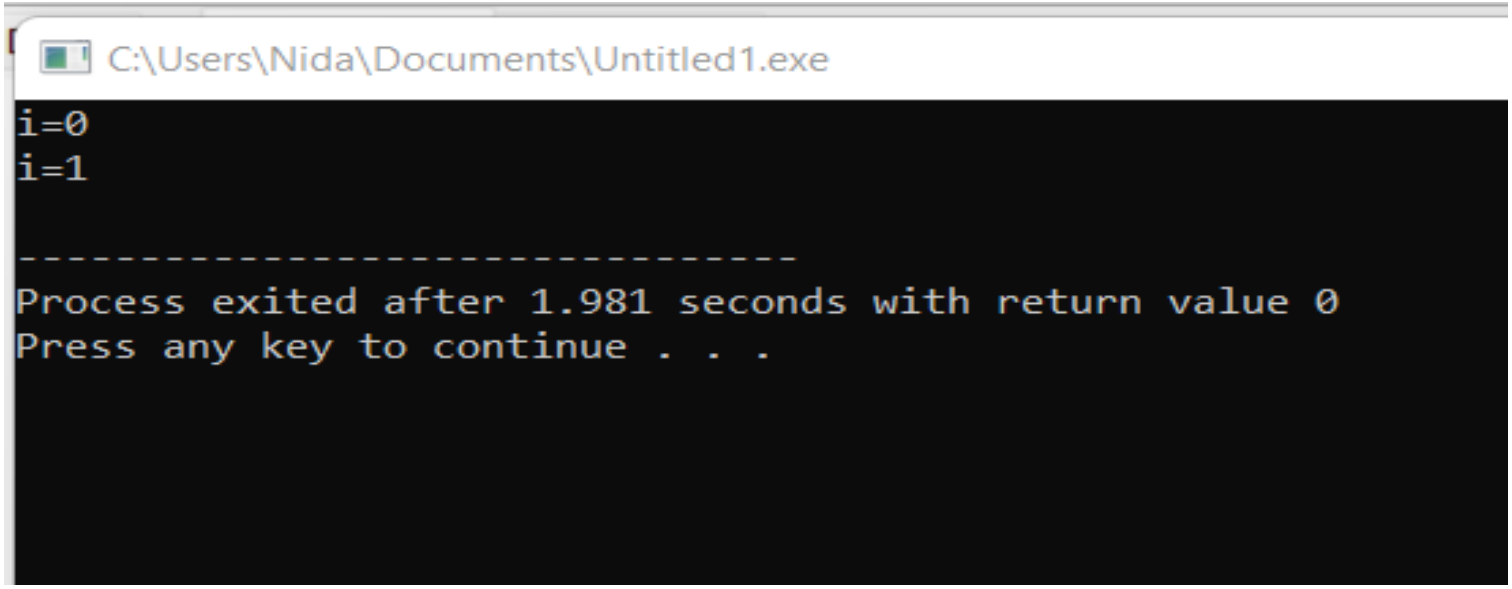
distinguish between post and prefix

- The operator symbol for both **prefix(++i)** and **postfix(i++)** are the same. Hence, we need two different function definitions to distinguish between them. This is achieved by passing a dummy int parameter in the postfix version.

For Postfix ++ Operator

```
#include<iostream>
using namespace std;
class Postfix{
      int i;
   public:
      Postfix(): i(0) {  }
      void operator ++(int)
        { i++; }
      void Display()
        { cout << "i=" << i << endl; }};
```

```
int main(){
    Postfix obj;
    obj.Display();
        obj.operator ++(4);//for this syntax you must have to pass a dummy value
        // you can also write obj++;
    obj.Display();
         return 0;}
```


C:\Users\Nida\Documents\Untitled1.exe
i=0
i=1

Process exited after 1.981 seconds with return value 0
Press any key to continue . . .

It should return old value as postfix rules

```
#include <iostream>
using namespace std;
class Postfix {
   int i;
public:
   Postfix() : i(0) {  }
   Postfix operator++(int) {
      Postfix temp = *this; // Store current state
      i++; // Increment current object
      return temp; // Return old state
   }

   void Display() {
      cout << "i = " << i << endl;
   }
};
```

Correct code for postfix

```
int main() {
    Postfix obj;
    obj.Display();

    Postfix obj2 = obj++;
    obj.Display();
    obj2.Display();  // Should still hold old value

    return 0;
}
```

- Prefix (++obj) allows chaining because it returns *this (a reference).
- Postfix (obj++) cannot allow chaining because it must return a temporary copy.

Since temp is a local object, it gets destroyed after the function returns.
So, if we write:
obj++.operator++();
This will not compile, because obj++ returns a temporary object, and we cannot modify a tempor

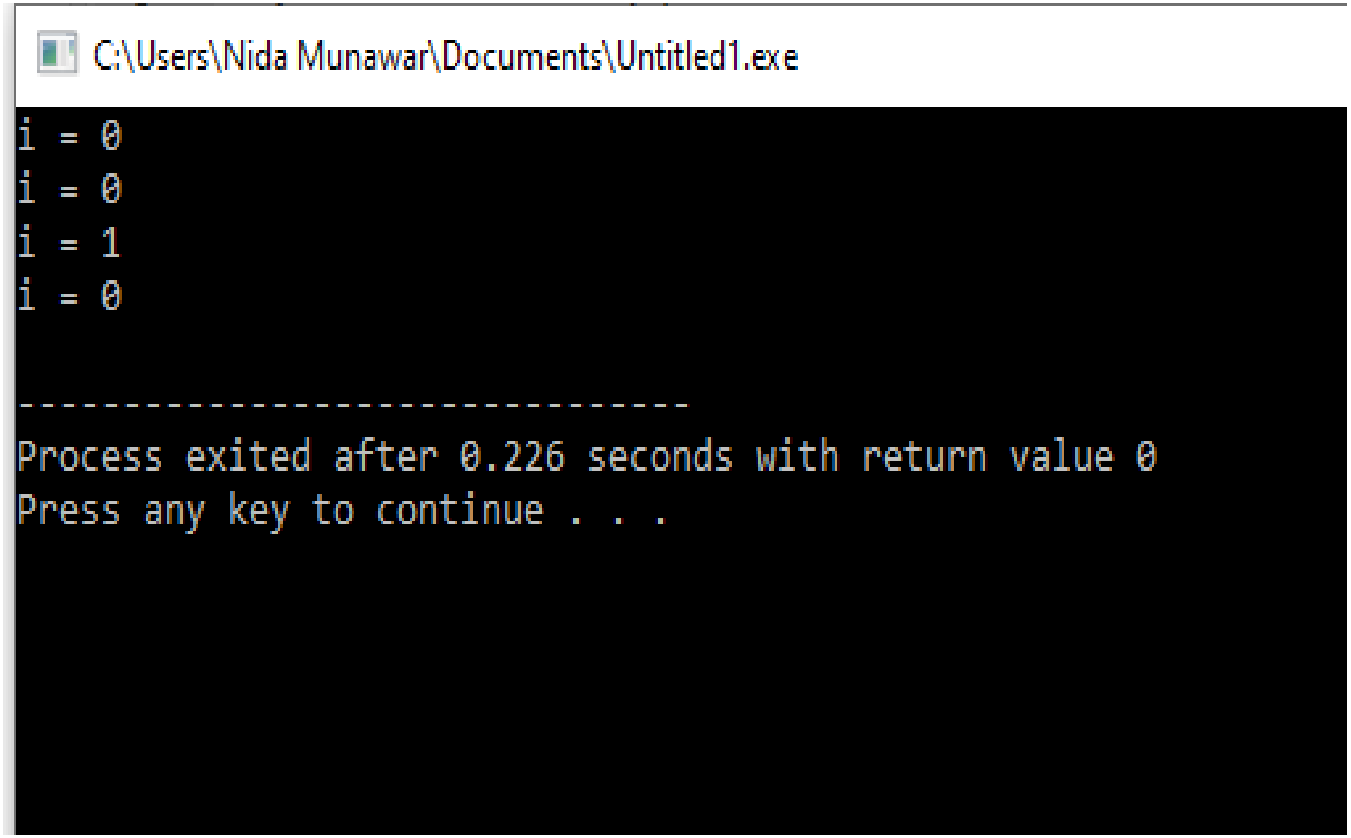
Output?

```
class Check{
    int i;
  public:
    Check(): i(0) {  }
    Check operator ++ (int){
        Check temp;
        temp.i = i++;
        return temp;}
void Display()
    { cout << "i = "<< i <<endl; }};
```

Output?

- int main(){
- Check obj, obj1;
- obj.Display();
- obj1.Display();
- obj1 = obj++;
- obj.Display();
- obj1.Display();
- return 0;}

Output


C:\Users\Nida Munawar\Documents\Untitled1.exe
i = 0
i = 0
i = 1
i = 0

Process exited after 0.226 seconds with return value 0
Press any key to continue . . .

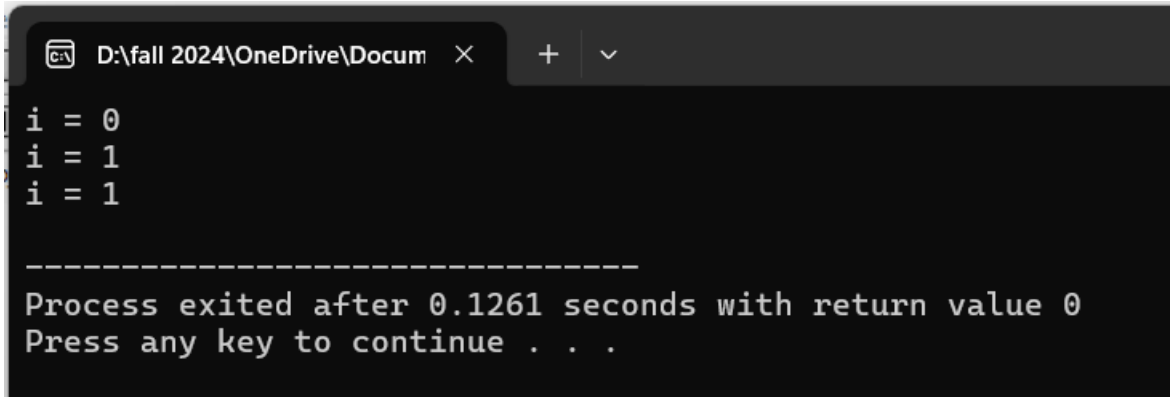
Why Not return *this; in Postfix?

```
#include <iostream>
using namespace std;
class Postfix {
    int i;
public:
    Postfix() : i(0) { }
    Postfix& operator++(int) {
        i++;  // Increment first
        return *this;   }
    void Display() {
        cout << "i = " << i << endl;
    }
};
```

```
int main() {
    Postfix obj;
    obj.Display();

    Postfix obj2 = obj++;
    obj.Display();
    obj2.Display();

    return 0;
}
```


D:\fall 2024\OneDrive\Docum
i = 0
i = 1
i = 1

Process exited after 0.1261 seconds with return value 0
Press any key to continue . . .

Restrictions on Operator Overloading

Operators that can be overloaded							
+	-	*	/	%	^	&	|
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	|=	<<	>>	>>=
<<=	==	!=	<=	>=	&&	||	++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

Operators that can be overloaded.

Operators that cannot be overloaded			
.	.*	::	?:

Operators that cannot be overloaded.

Assignment Operator =

The assignment operator has a signature like this:

```
class MyClass {
public:
MyClass & operator=(const MyClass &rhs);
} ;
int main(){
MyClass a, b; ...
b = a; // Same as b.operator=(a);
}
```

Notice that the = operator takes a const-reference to the right hand side of the assignment.
The reason for this should be obvious, since we don't want to change that value; we only want to change what's on the left hand side.
Also, you will notice that a reference is returned by the assignment operator.

This is to allow **operator chaining**. You typically see it with primitive types, like this:

operator chaining

- **operator chaining** with primitive types, like this:
- int a, b, c, d, e;
- a = b = c = d = e = 42;
- This is interpreted by the compiler as:
- a = (b = (c = (d = (e = 42))));
- In other words, assignment is right-associative. The last assignment operation is evaluated first, and is propagated leftward through the series of assignments. Specifically:
- e = 42 assigns 42 to e, then returns e as the result
- The value of e is then assigned to d, and then d is returned as the result
- The value of d is then assigned to c, and then c is returned as the result etc.

operator chaining

- Now, in order to support operator chaining, the assignment operator must return some value. The value that should be returned is a reference to the left-hand side of the assignment.

operator chaining with =

```
#include <iostream>
using namespace std;
class MyClass {
private:
    int value;  // Example member variable
public:
    // Constructor
    MyClass(int val = 0) : value(val) {}

MyClass& operator=(const MyClass &rhs) {
            this->value = rhs.value;     }
        return *this;  // Return reference to the current object
    }
    void display() const {
        cout << "Value: " << value << endl;
    }
};
```

```
int main() {
    MyClass a(10), b(20) , c(30);
    cout << "Before assignment:" << endl;
    a.display();
    b.display();
    c.display();
    c = b = a;
    cout << "After assignment:" << endl;
    a.display();
    b.display();
    c.display();
    return 0;
}
```

Explanation:

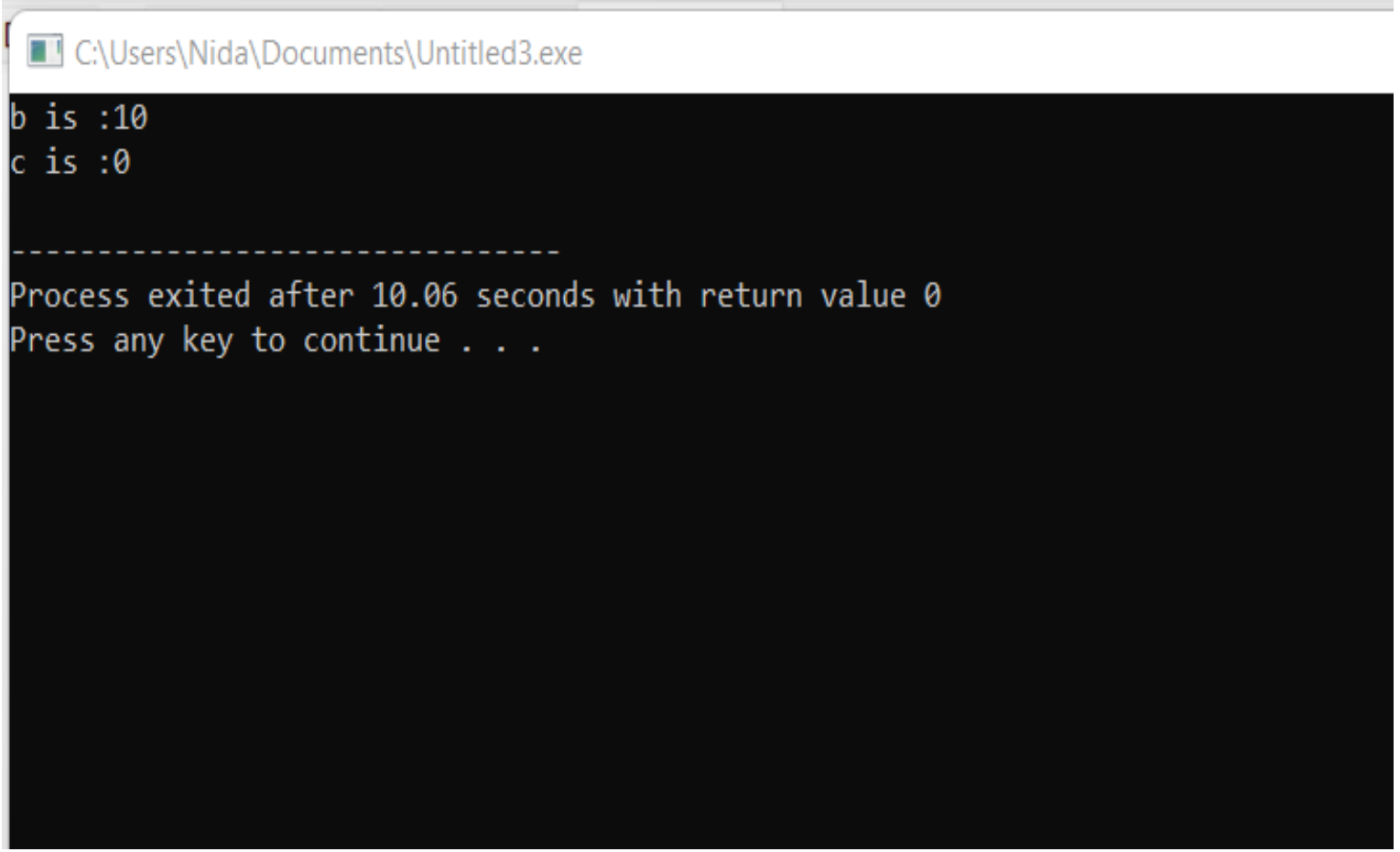
c = b = a; // Same as b.operator=(a), then c.operator=(b);

1. Right-to-left evaluation:
   - b = a → Calls b.operator=(a), copying a into b.
   - c = b → Calls c.operator=(b), copying b (which is now equal to a) into c.
2. Since operator= returns *this (a reference to the current object), it allows chaining (c = b = a works as (c = (b = a))).

operator chaining

```
#include<iostream>
using namespace std;
int main(){
        int a = 10 , b , c = 0;
        (b=c)=a;
        cout << "b is :" << b << endl;
        cout << "c is :" << c << endl;
}
```

operator chaining



operator chaining

```
#include<iostream>
using namespace std;
class chain {
  private:
    int var;
  public:
chain(){
       var = 0; }
chain(int val){
       var = val;
       }
void display() {
 cout << "var: "<< var <<endl;     }
```

operator chaining

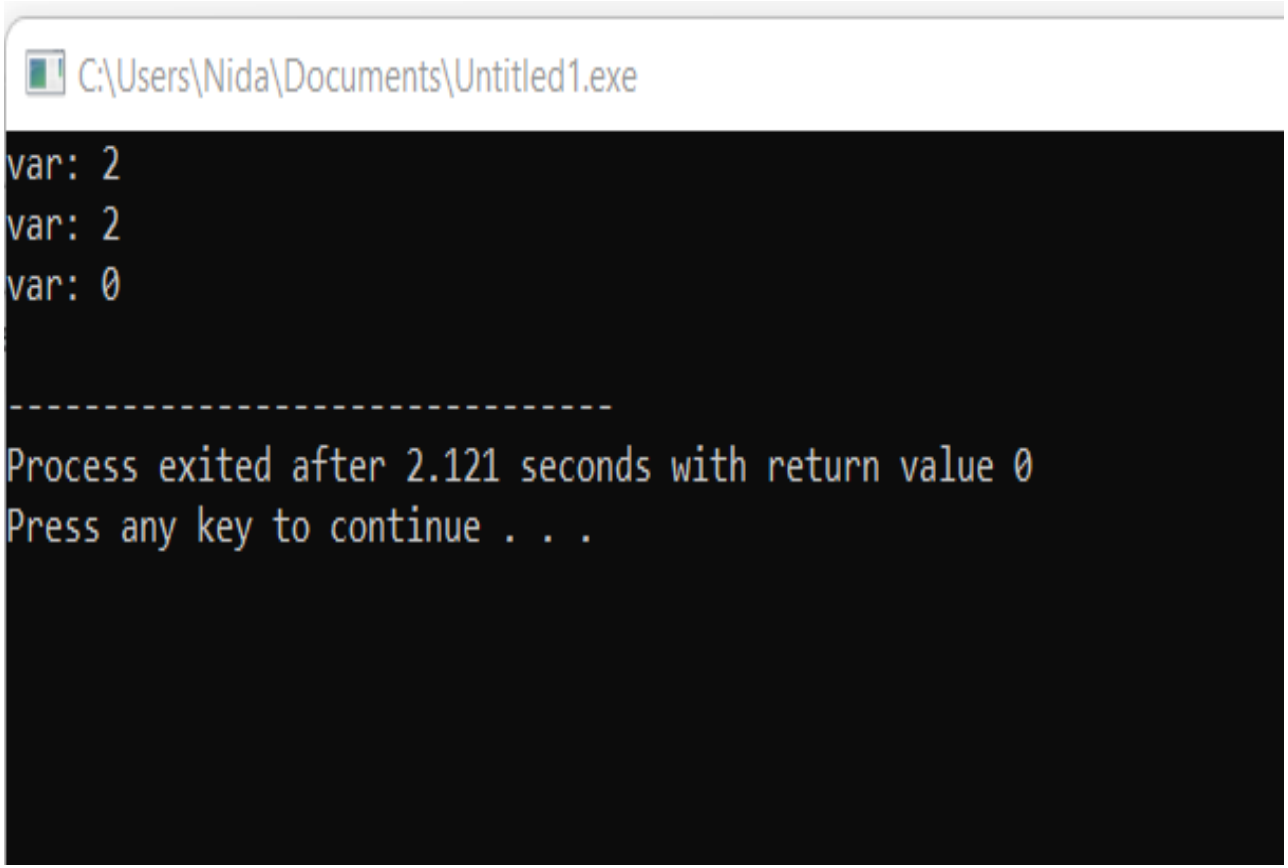
```
// overloaded  (=) operator
     chain& operator= (chain rhs) {
        this->var=rhs.var; // this pointer is
optional here
        return *this; }};
```

operator chaining

```
int main() {
        chain c1(2);
        chain c2 , c3;
         (c2=c3)=c1;
       c2.display();
       c1.display();
                c3.display();
        return 0;}
```

- Step 1: c2 = c3 → c2.operator=(c3)
- Copies c3.var into c2.var.
- Returns *this (which is c2).
- Step 2: (c2 = c3) = c1 → Now, c2 = c1
- Copies c1.var into c2.var.

operator chaining



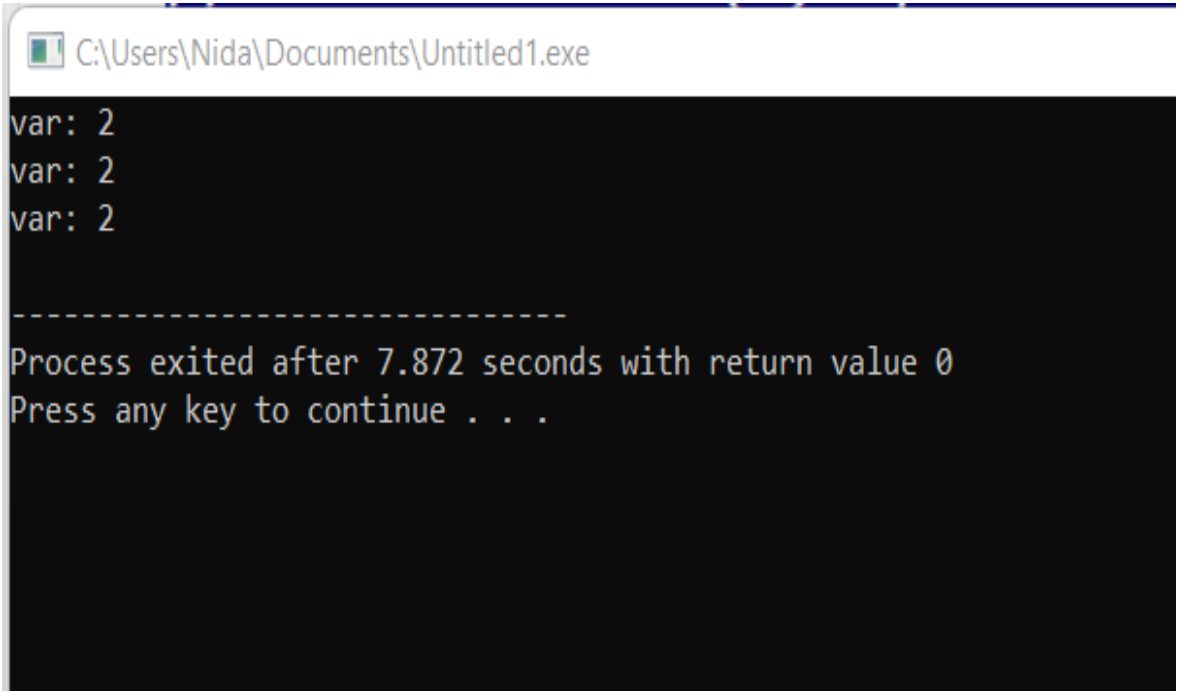
operator chaining

If you are returning by value

Works fine when there is no parenthesis

```
// overloaded  (=) operator
      chain operator= (chain rhs) {
         this->var=rhs.var; // this pointer is optional here
         return *this; }};
int main() {
         chain c1(2);
         chain c2 , c3;
          c2=c3=c1;
        c2.display();
        c1.display();
                    c3.display();
          return 0;}
```

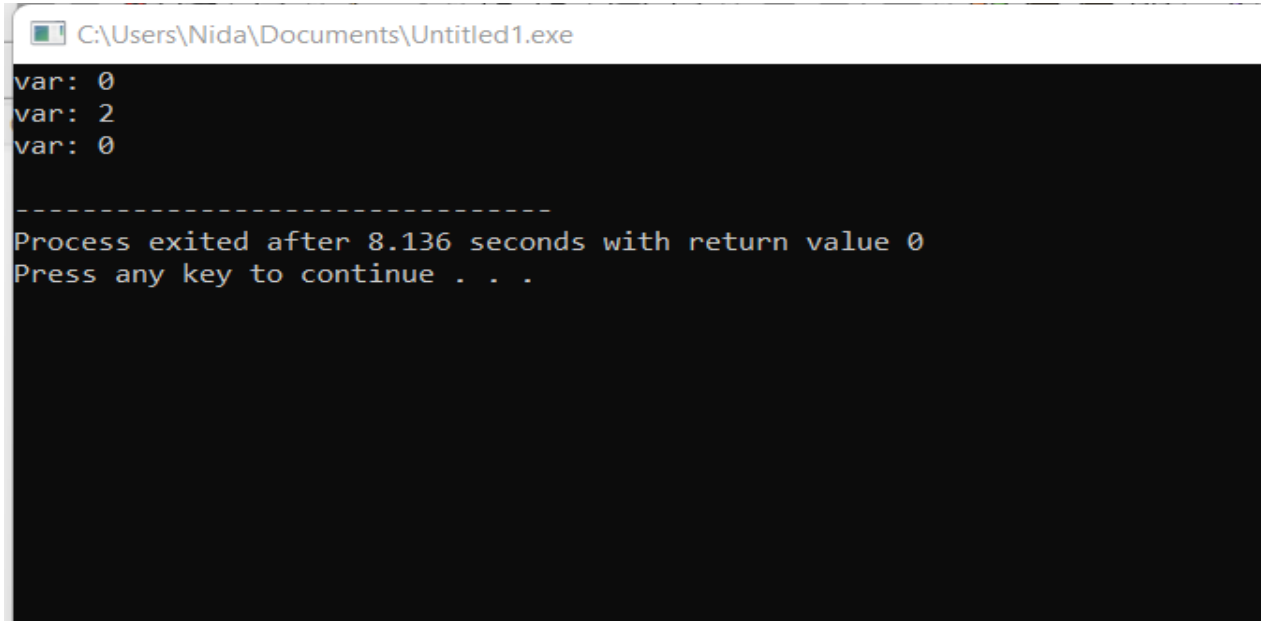
operator chaining



If you are returning by value

```
chain operator= (chain rhs) {
        this->var=rhs.var;
        return *this; }
int main() {
            chain c1(2);
            chain c2 , c3;
           (c2=c3)=c1;
        c2.display();
        c1.display();
            c3.display();
         return 0;}
```

operator chaining



Operator chaining is not possible by returning value you must return reference

Issue: Returning by Value (chain operator=(chain rhs))

Your assignment operator is returning a copy instead of a reference.

(c2 = c3) = c1;

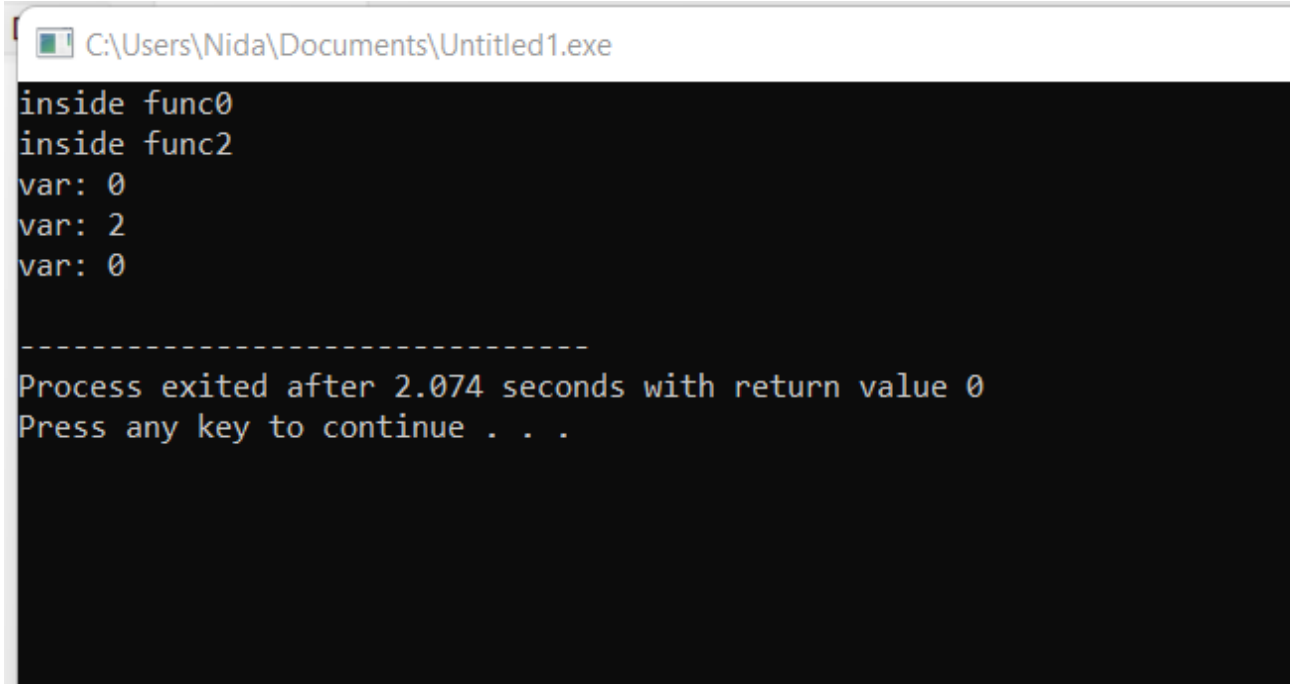
c2 = c3 returns a temporary object (copy) of c2.

- Then, = c1 is applied to this temporary copy, not c2 itself.
- This means c2 is not affected by c1, and its value remains from c3.

In Pass by value changes only reflect inside function scope not in main

```
#include<iostream>
using namespace std;
class chain {
  private:
    int var;
  public:
chain(){
      var = 0; }
chain(int val){
      var = val;
      }
void display() {
 cout << "var: "<< var <<endl;    }

chain operator= (chain rhs) {
      this->var=rhs.var;
      cout << "inside func" << rhs.var<< endl;
      return *this; }
    };
int main() {
            chain c1(2);
            chain c2 , c3;
        (c2=c3)=c1;
      c2.display();
      c1.display();
            c3.display();
       }
```


C:\Users\Nida\Documents\Untitled1.exe
inside func0
inside func2
var: 0
var: 2
var: 0

Process exited after 2.074 seconds with return value 0
Press any key to continue . . .

operator chaining

- So, for the hypothetical MyClass assignment operator, you would do something like this:
- // Take a const-reference to the right-hand side of the assignment.
- // Return a non-const reference to the left-hand side.
- MyClass& MyClass::operator=(const MyClass &rhs) {
- ... // Do the assignment operation!
-
- return *this; // Return a reference to myself.
- }
-

operator chaining

- Remember, this is a pointer to the object that the member function is being called on. Since a = b is treated as a.operator=(b), you can see why it makes sense to return the object that the function is called on; object a is the left-hand side.
- But, the member function needs to return a reference to the object, not a pointer to the object. So, it returns *this, which returns what this points at (i.e. the object), not the pointer itself. (In C++, instances are turned into references, and vice versa, pretty much automatically, so even though *this is an instance, C++ implicitly converts it into a reference to the instance.)

For += operator

```
Vector& operator += (const Vector &ob)
{
        x += ob.x;
        y += ob.y;
        return *this;
}
```

For Unary - operator (Negation)

```
Vector operator – ( ) const
{
        Vector temp;
        temp.x = -x;
        temp.y = -y;
        return temp;
}
```

Next Lecture

- Function Overriding